

Sri Lanka Institute of Information Technology

BSc (Hons) Information Technology
Specializing in Data Science



IT3021 – Data Warehousing and Business Intelligence

Year 03 Semester 01

Assignment 1

Student name – THEAVER S.T.T.

Student ID – IT23563032

Table of Contents

1. Dataset Selection

- 1.1 Introduction
- 1.2 Dataset Overview
- 1.3 Source Types and Formats
- 1.4 Coverage and Realism
- 1.5 Suitability for Data Warehousing and BI
- 1.6 Entity Relationship Diagram (ERD)

2. Preparation of Data Sources

- 2.1 Source 1 — Superstore_SourceDB (SQL Server)
- 2.2 Source 2 — Customers_Source.xlsx (Excel)
- 2.3 Source 3 — Products_Source.xlsx (Excel)

3. Solution Architecture.

- 3.1 Architecture Diagram
- 3.2 Component Descriptions

4. Data Warehouse Design & Development

- 4.1 Dimensional Model Overview
- 4.2 Relational Diagram- Star Schema Diagram
- 4.3 Design Assumptions
- 4.4 SQL Implementation

5. ETL Development

- 5.1 SSIS Packages Overview
- 5.2 Connection Managers
- 5.3 Package 1 — Superstore_Load_Staging.dtsx
- 5.4 Package 2 — Superstore_Load_DW.dtsx
 - 5.4a Load DimLocation
 - 5.4b Load DimShipMode
 - 5.4c Load DimProduct
 - 5.4d Load DimCustomer (SCD Type 2)
 - 5.4e Load FactSales

6. ETL Development – Accumulating fact tables

- 6.1 Accumulating Fact Concept
- 6.2 OrderCompletions.csv
- 6.3 Package 3 — Superstore_UpdateFact.dtsx
- 6.4 Validation Results

7. Conclusion

Introduction

This report documents the complete design and implementation of a Data Warehouse and Business Intelligence solution built for the **Superstore Retail Sales** dataset as part of IT3021 — Data Warehousing and Business Intelligence, Year 3 Semester 1/2, 2026.

The Superstore dataset represents a US-based retail company operating across four geographic regions — Central, East, South, and West — selling products in three main categories: Furniture, Office Supplies, and Technology. The dataset contains 9,994 order line items spanning five years (2014–2018) across 793 unique customers and 1,862 distinct products. It was selected because it provides a rich, realistic transactional dataset with sufficient volume, multiple entity types, and natural hierarchies to demonstrate all required data warehousing concepts including dimensional modelling, ETL integration, Slowly Changing Dimensions, and accumulating fact tables.

The data was structured across three heterogeneous source types to satisfy the multiple-source requirement: a SQL Server relational database (Superstore_SourceDB) containing transactional order and order detail records, a Microsoft Excel file (Customers_Source.xlsx) containing customer master data exported from a CRM system, and a second Microsoft Excel file (Products_Source.xlsx) containing product catalogue and category hierarchy data. This deliberate separation simulates a real enterprise integration scenario involving database, file-based, and spreadsheet data sources.

The solution implements the full DW & BI pipeline across four layers:

Source Systems → Superstore_Staging → Superstore_DW → BI Layer (Assignment 2)

The data warehouse (Superstore_DW) is implemented as a Star Schema consisting of one central fact table (FactSales) and five dimension tables: DimCustomer (SCD Type 2), DimProduct (SCD Type 1), DimLocation, DimDate, and DimShipMode. The ETL process is implemented using SQL Server Integration Services (SSIS) across three packages: Superstore_Load_Staging.dtsx, Superstore_Load_DW.dtsx, and Superstore_UpdateFact.dtsx. Task 6 extends the solution with an accumulating snapshot fact pattern to track order lifecycle from creation to completion.

This report documents each step with screenshots, SQL scripts, and design justifications in the order they were implemented.

Step 1: Data Set Selection

1.1 Introduction

The Sample Superstore dataset, a widely referenced **retail analytics dataset**, was selected as the OLTP source for this assignment. This dataset represents the core transactional operations of a US-based retail company, capturing sales orders, customer demographics, product catalogue information, and shipping logistics across five years of business activity.

It is a **novel and practical real-world scenario** that has not been used in any practical sessions and does not replicate the dataset used in any lab sessions. The dataset is highly suitable for demonstrating data warehousing concepts, ETL processing, and business intelligence reporting across multiple business dimensions.

1.2 Data Set Overview

The dataset covers retail sales transactions spanning five years (2014–2018) across three source types. It contains 9,994 transactional line-item records with 21 attributes covering customers, products, geography, time, and shipping, making it highly suitable for data warehousing and BI analytics.

Key entities in the dataset:

- **Customers (Excel File):** Customer master data including name, customer segment classification (Consumer, Corporate, Home Office), geographic location (city, state, postal code, region), and country.
- **Products (Excel File):** Product catalogue with category, sub-category, and product name supporting a 3-level hierarchy (Category → Sub-Category → Product) for drill-down reporting.
- **Transactions (SQL Server Database):** Core transactional tables — Orders and OrderDetails — containing order IDs, dates, ship mode, and line-item sales values including Sales, Quantity, Discount, and Profit.

1.3 Source Types and Formats

To simulate a realistic enterprise environment, the dataset was split across **three different source types**:

Source File / Table	Format	Description
Superstore_SourceDB (Orders + OrderDetails)	SQL Server DB	Core transactional data: OrderID, OrderDate, ShipDate, ShipMode, CustomerID, ProductID, Quantity, Sales, Discount, Profit
Customers_Source.xlsx	Microsoft Excel	Customer master data: CustomerID, CustomerName, Segment, City, State, PostalCode, Country, Region
Products_Source.xlsx	Microsoft Excel	Product catalogue with hierarchy: ProductID, ProductName, CategoryID, CategoryName, SubCategoryID, SubCategoryName

Table -Source types and formats

1.4 Coverage and Realism

The dataset includes sufficient variety and depth to support key data warehousing requirements:

- **Multi-year coverage:** 2014–2018, 9,994 records enable trend analysis and YoY comparisons.
- **Diverse customer segments:** Consumer, Corporate, Home Office supports customer lifetime value and segment analysis.
- **Geographic diversity:** 49 US states across 4 regions allows regional and state-level performance analysis.
- **Product hierarchy:** 3 categories, 17 sub-categories, 1,862 SKUs supports hierarchical reporting.
- **Financial data:** Sales, Quantity, Discount, Profit per line enables profitability and revenue analysis.
- **Shipping & fulfilment:** 4 ship modes support logistics performance analysis.
- **Multiple source types:** SQL Server + Excel files simulates heterogeneous enterprise environment for ETL integration.

1.5 Suitability for Data Warehousing and BI

The dataset was evaluated against all major assignment requirements:

Requirement	How It Is Satisfied
OLTP Structure	Normalized transactional records split across three source types, each with well-defined entity relationships and no pre-aggregated fact/dimension structures.
Novel Scenario	US retail sales environment across 5 years — not used in lab sessions and not the AdventureWorks dataset.
At least one year of data	Dataset spans five years (2014–2018) with 9,994 transaction records, providing a rich timeline for trend and seasonal analysis.
Multiple source types (≥ 2)	SQL Server database (orders) + Excel file (customers) + Excel file (products) — three distinct source types.
Complex ETL support	Includes SCD Type 2 on DimCustomer (customer segment changes), derived date dimension, null handling, data type conversions, and accumulating snapshot facts.
Warehouse-ready dimensional modelling	Supports a star schema with one central fact table (FactSales) and five dimension tables: DimCustomer, DimProduct, DimLocation, DimDate, and DimShipMode.
SSAS cube and reporting compatibility	Multiple hierarchies: Category → Sub-Category → Product; Region → State → City; Year → Quarter → Month. Measures include Sales, Profit, Quantity, and Discount.

Table — Suitability evaluation for Data Warehousing and BI

Note: The source dataset is a single flat CSV file. For this assignment, it was deliberately split into three source types, a SQL Server database and two Excel files, to simulate a realistic heterogeneous enterprise environment and demonstrate multi-source ETL integration as required by the assignment specification.

1.6 Entity Relationship Diagram

The following ERD illustrates the logical relationships between the source data entities prior to transformation.

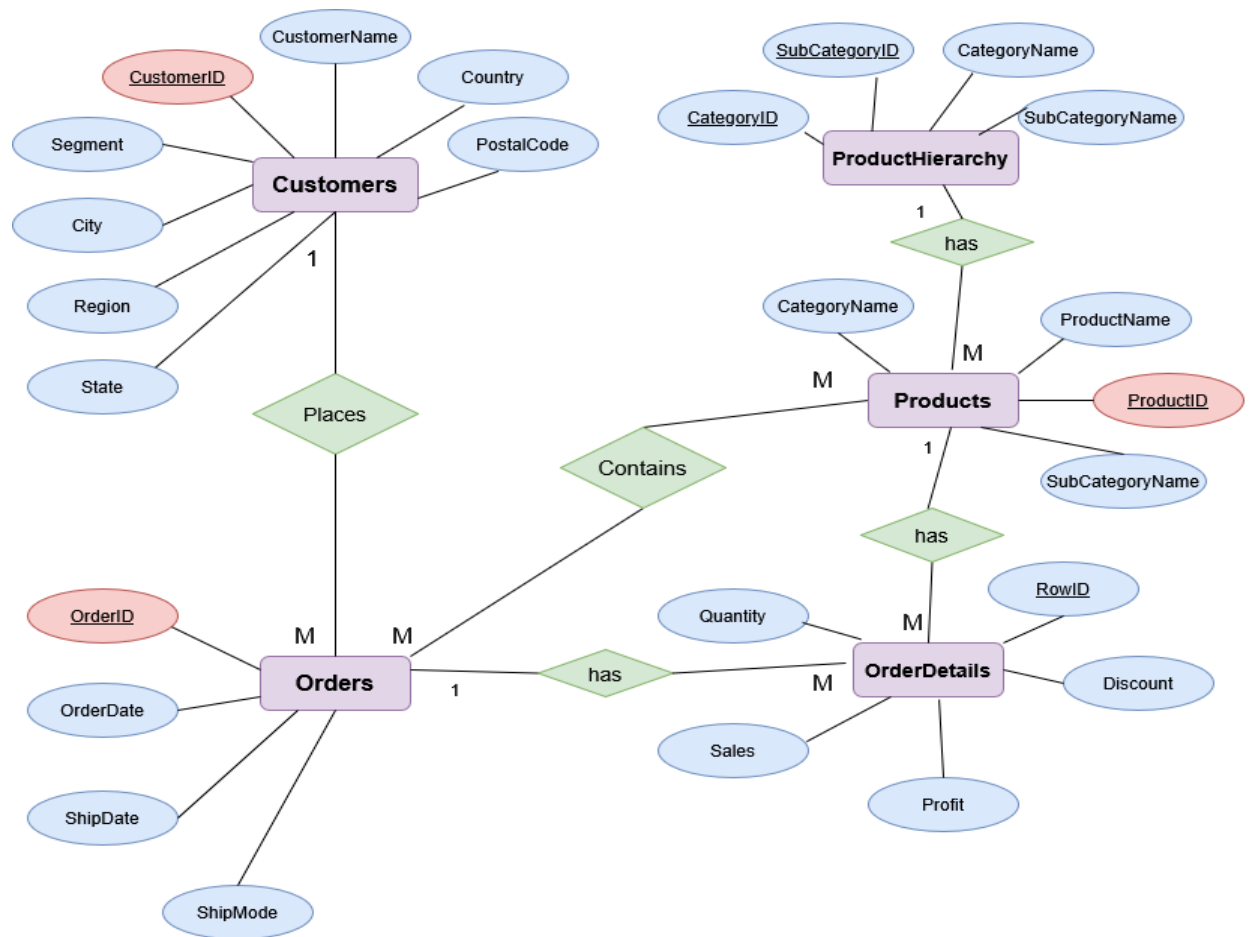


Figure : ER Diagram of the Superstore Source Dataset

Step 2: Source Data Description

To simulate a realistic enterprise data warehousing environment, the source data for this project is extracted from three distinct systems: a SQL Server database, and two Microsoft Excel files. These heterogeneous sources demonstrate the ETL process's ability to handle varying data formats while ensuring integration accuracy and consistency.

2.1 SQL Server Database Source

The primary source system is a SQL Server database named Superstore_SourceDB. This database contains the core transactional operations of the retail company split into two normalized tables: Orders and OrderDetails.

Orders Table (5,009 rows) — contains the following fields:

- **OrderID** (NVARCHAR(20), PK)
- **OrderDate** (DATE)
- **ShipDate** (DATE)
- **ShipMode** (NVARCHAR(20))
- **CustomerID** (NVARCHAR(10), FK)

OrderDetails Table (9,994 rows) — contains the following fields:

- **RowID** (INT, PK)
- **OrderID** (NVARCHAR(20), FK)
- **ProductID** (NVARCHAR(20), FK)
- **Quantity** (INT)
- **Sales** (DECIMAL(12,4))
- **Discount** (DECIMAL(5,4))
- **Profit** (DECIMAL(12,4))

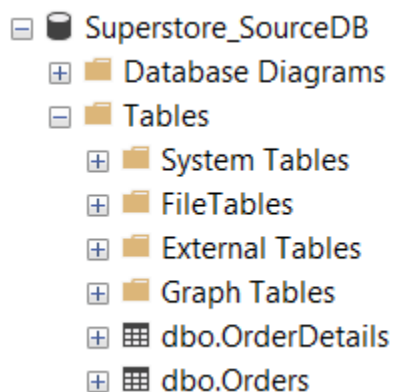


Figure : Superstore_SourceDB in SSMS Object Explorer

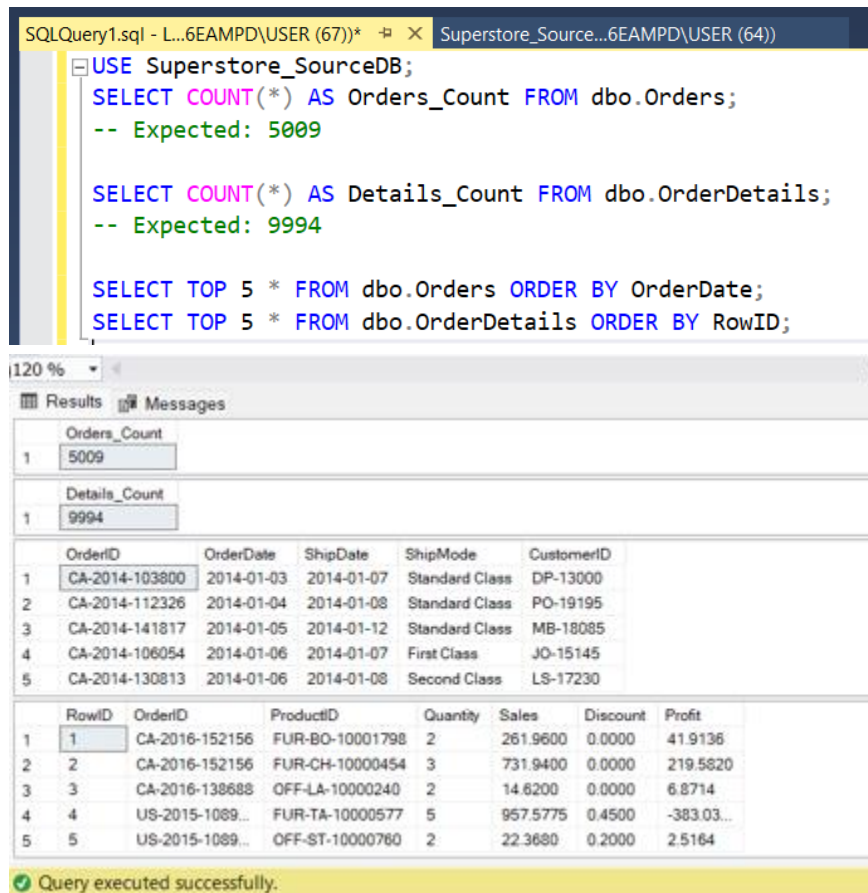


Figure: Row counts for Orders and OrderDetails tables , Sample data from the Orders source table

2.2 Excel File Source – Customers

The Customers_Source.xlsx file contains customer master data exported from a CRM system. This file simulates data originating from a non-database system. It contains 793 unique customer records with the following structure:

Customers_Source.xlsx (793 rows) — contains the following fields:

- **CustomerID** (NVARCHAR(10), PK)
- **CustomerName** (NVARCHAR(50))
- **Segment** (NVARCHAR(15))
- **City** (NVARCHAR(30)) - 3rd level of the geographic hierarchy
- **State** (NVARCHAR(30)) - 2nd level of the geographic hierarchy
- **PostalCode** (NVARCHAR(10))
- **Country** (NVARCHAR(20)) — Currently United States only
- **Region** (NVARCHAR(15)) — Central, East, South, West (*1st level of geographic hierarchy*)

	A	B	C	D	E	F	G	H	I
1	CustomerID	CustomerName	Segment	City	State	PostalCode	Country	Region	
2	AA-10315	Alex Avila	Consumer	Minneapolis	Minnesota	55407	United States	Central	
3	AA-10375	Allen Arnold	Consumer	Mesa	Arizona	85204	United States	West	
4	AA-10480	Andrew Allen	Consumer	Concord	North Carolina	28027	United States	South	
5	AA-10645	Anna Andreadi	Consumer	Chester	Pennsylvania	19013	United States	East	
6	AB-10015	Aaron Bergman	Consumer	Seattle	Washington	98103	United States	West	
7	AB-10060	Adam Bellavance	Home Office	New York City	New York	10009	United States	East	
8	AB-10105	Adrian Barton	Consumer	Phoenix	Arizona	85023	United States	West	
9	AB-10150	Aimee Bixby	Consumer	Long Beach	New York	11561	United States	East	
10	AB-10165	Alan Barnes	Consumer	Los Angeles	California	90036	United States	West	
11	AB-10255	Alejandro Ballentine	Home Office	Lorain	Ohio	44052	United States	East	
12	AB-10600	Ann Blume	Corporate	Tucson	Arizona	85705	United States	West	
13	AC-10420	Alyssa Crouse	Corporate	San Francisco	California	94122	United States	West	
14	AC-10450	Amy Cox	Consumer	Minneapolis	Minnesota	55407	United States	Central	
15	AC-10615	Ann Chong	Corporate	New York City	New York	10009	United States	East	
16	AC-10660	Anna Chung	Consumer	Huntsville	Texas	77340	United States	Central	
17	AD-10180	Alan Dominguez	Home Office	Houston	Texas	77041	United States	Central	
18	AF-10870	Art Ferguson	Consumer	College Station	Texas	77840	United States	Central	
19	AF-10885	Art Foster	Consumer	Louisville	Kentucky	40214	United States	South	

Figure: Customers_Source.xlsx open in Excel showing all 8 column headers and first few data rows.

2.3 Excel File Source – Products

The Products_Source.xlsx file contains the product catalogue data in Microsoft Excel format with two sheets. This file simulates integration from a product management system export. It contains 1,862 distinct product SKUs organized into a 3-level hierarchy.

Sheet 1 – ProductHierarchy (17 rows): — contains the following fields:

- **CategoryID** - (INT)
- **CategoryName** -(NVARCHAR(30))
- **SubCategoryID** -(INT)
- **SubCategoryName** - (NVARCHAR(30))

	A	B	C	D
1	CategoryID	CategoryName	SubCategoryID	SubCategoryName
2	1	Furniture	1	Bookcases
3	1	Furniture	2	Chairs
4	1	Furniture	6	Furnishings
5	1	Furniture	4	Tables
6	2	Office Supplies	10	Appliances
7	2	Office Supplies	7	Art
8	2	Office Supplies	9	Binders
9	2	Office Supplies	13	Envelopes
10	2	Office Supplies	14	Fasteners
11	2	Office Supplies	3	Labels
12	2	Office Supplies	11	Paper
13	2	Office Supplies	5	Storage
14	2	Office Supplies	15	Supplies
15	3	Technology	12	Accessories
16	3	Technology	17	Copiers
17	3	Technology	16	Machines
18	3	Technology	8	Phones

Figure: Products_Source.xlsx open in Excel showing the Products Hierarchy sheet 2 with 17 rows

Sheet 2 – Products (1,862 rows) — contains the following fields:

- **ProductID** –(NVARCHAR(20) PK)
- **ProductName**–(NVARCHAR(200))
- **CategoryID** –(INT FK)
- **CategoryName** –(NVARCHAR(30))
- **SubCategoryID** – (INT FK)
- **SubCategoryName** –(NVARCHAR(30))

1	ProductID	ProductName	CategoryID	CategoryName	SubCategoryID	SubCategoryName
2	FUR-BO-10000112	Bush Birmingham Collection Bookcase, Dark Cherry	1	Furniture		1 Bookcases
3	FUR-BO-10000330	Sauder Camden County Barrister Bookcase, Planked Cherry Finish	1	Furniture		1 Bookcases
4	FUR-BO-10000362	Sauder Inglewood Library Bookcases	1	Furniture		1 Bookcases
5	FUR-BO-10000468	O'Sullivan 2-Shelf Heavy-Duty Bookcases	1	Furniture		1 Bookcases
6	FUR-BO-10000711	Hon Metal Bookcases, Gray	1	Furniture		1 Bookcases
7	FUR-BO-10000780	O'Sullivan Plantations 2-Door Library in Landvery Oak	1	Furniture		1 Bookcases
8	FUR-BO-10001337	O'Sullivan Living Dimensions 2-Shelf Bookcases	1	Furniture		1 Bookcases
9	FUR-BO-10001519	O'Sullivan 3-Shelf Heavy-Duty Bookcases	1	Furniture		1 Bookcases
10	FUR-BO-10001567	Bush Westfield Collection Bookcases, Dark Cherry Finish, Fully Assembled	1	Furniture		1 Bookcases
11	FUR-BO-10001601	Sauder Mission Library with Doors, Fruitwood Finish	1	Furniture		1 Bookcases
12	FUR-BO-10001608	Hon Metal Bookcases, Black	1	Furniture		1 Bookcases
13	FUR-BO-10001619	O'Sullivan Cherrywood Estates Traditional Bookcase	1	Furniture		1 Bookcases
14	FUR-BO-10001798	Bush Somerset Collection Bookcase	1	Furniture		1 Bookcases
15	FUR-BO-10001811	Atlantic Metals Mobile 5-Shelf Bookcases, Custom Colors	1	Furniture		1 Bookcases
16	FUR-BO-10001918	Sauder Forest Hills Library with Doors, Woodland Oak Finish	1	Furniture		1 Bookcases
17	FUR-BO-10001972	O'Sullivan 4-Shelf Bookcase in Odessa Pine	1	Furniture		1 Bookcases
18	FUR-BO-10002202	Atlantic Metals Mobile 2-Shelf Bookcases, Custom Colors	1	Furniture		1 Bookcases
19	FUR-BO-10002208	Bush Saratoga Collection 5-Shelf Bookcase, Hanover Cherry, *Special Order	1	Furniture		1 Bookcases

Figure: Products_Source.xlsx open in Excel showing the Products sheet 2 with 1,862 rows

The use of three distinct source formats (SQL Server, Excel – Customers, Excel – Products) satisfies the assignment requirement for at least two data source types and enables the ETL pipeline to demonstrate connection manager configuration, Excel file import, and OLE DB extraction within SSIS.

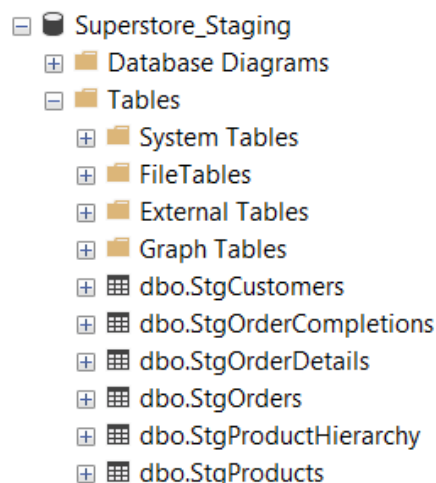


Figure: SSMS Object Explorer showing Superstore_Staging database with all 6 staging tables visible

Step 3: Solution Architecture

A high-level Data Warehouse and Business Intelligence solution architecture was designed to support the integration, transformation, and analysis of Superstore retail sales data. The architecture reflects a realistic enterprise BI pipeline with clearly defined layers, starting from operational data sources and ending with user-facing analytical tools

3.1 Architectural Diagram

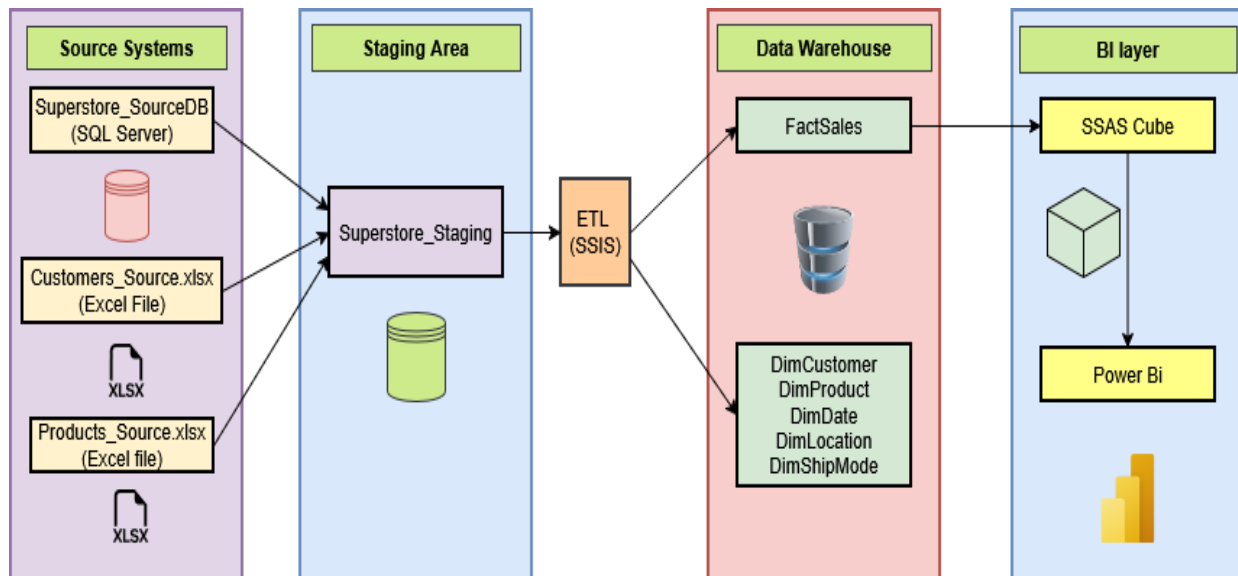


Figure : High-Level DW & BI Solution Architecture

3.2 Component Summary

Layer	Component	Description
Source Systems	Superstore_SourceDB (SQL Server)	Primary OLTP source database containing the Orders (5,009 rows) and OrderDetails (9,994 rows) tables with normalized transactional records.
Source Systems	Customers_Source.xlsx (Excel)	Excel export containing 793 unique customer records with demographic and geographic data, simulating a CRM system export.
Source Systems	Products_Source.xlsx (Excel)	Product catalogue Excel file containing 1,862 unique product SKUs across 3 categories and 17 sub-categories, with a built-in hierarchy.
Staging Area	Superstore_Staging	Intermediate SQL Server database where data from all three sources is loaded, cleansed, validated, and standardized before DW loading.
ETL Process	SSIS (SQL Server Integration Services)	Three SSIS packages handle extraction, transformation (SCD Type 2, null handling, derived columns), and loading. Executed sequentially for dependency management.
Data Warehouse	Superstore_DW	Star schema in SQL Server with FactSales as the central fact table and five dimension tables: DimCustomer, DimProduct, DimLocation, DimDate, and DimShipMode.
BI Layer	SSAS Cube	Multidimensional cube enabling OLAP queries with drill-down by geography, product hierarchy, and time dimensions.
BI Layer	Power BI	Visual dashboards for sales performance, profitability analysis, customer segment reporting, and regional distribution.

Table — DW & BI Solution Architecture Component Summary

Step 4: Data Warehouse Design & Development

A star schema-based dimensional model was developed for the Superstore Retail Sales system. This design provides an analytical foundation for exploring sales performance, customer behaviour, product trends, geographic patterns, and shipping efficiency over time. The schema includes one central fact table (FactSales) and five dimension tables.

4.1 Dimensional Model Description

Table	Type	Description
FactSales	Fact Table	One row per transaction line item (OrderDetails grain). Measures: Sales, Quantity, Discount, Profit. Includes accumulating fact fields: accm_txn_create_time, accm_txn_complete_time, txn_process_time_hours.
DimCustomer	Dimension (SCD 2)	Customer demographics: CustomerName, Segment, location reference. Implements SCD Type 2 to track historical changes in Customer Segment using StartDate, EndDate, and IsCurrent columns.
DimProduct	Dimension (SCD 1)	Product metadata including ProductName, SubCategoryName, CategoryName, supporting the 3-level hierarchy Category → Sub-Category → Product for drill-down and SSAS cube hierarchies.
DimLocation	Dimension (SCD 1)	Geographic location data enabling 3-level drill-down: Region → State → City. Covers 4 regions, 49 states, and multiple cities.
DimDate	Dimension	Date reference table enabling analysis by Day, Week, Month, Quarter, and Year. Pre-populated for 2014–2018(1826 rows) to cover all order and ship dates in the source data. Used as both OrderDateKey and ShipDateKey in FactSales.
DimShipMode	Dimension (SCD 1)	Shipping mode dimension: Standard Class, Second Class, First Class, Same Day. Enables logistics and fulfilment performance analysis.

Table — Dimensional model description for the Superstore Data Warehouse

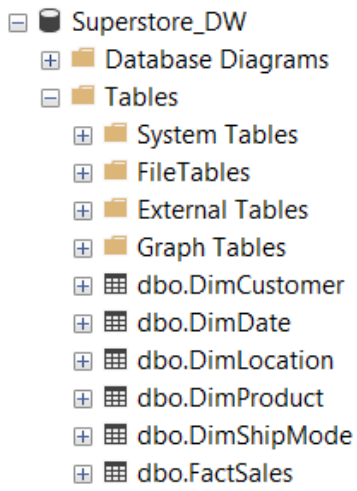


Figure : Superstore_DW tables in Object Explorer

4.2 Relational Diagram - Star Schema Diagram

The star schema ER diagram is presented below

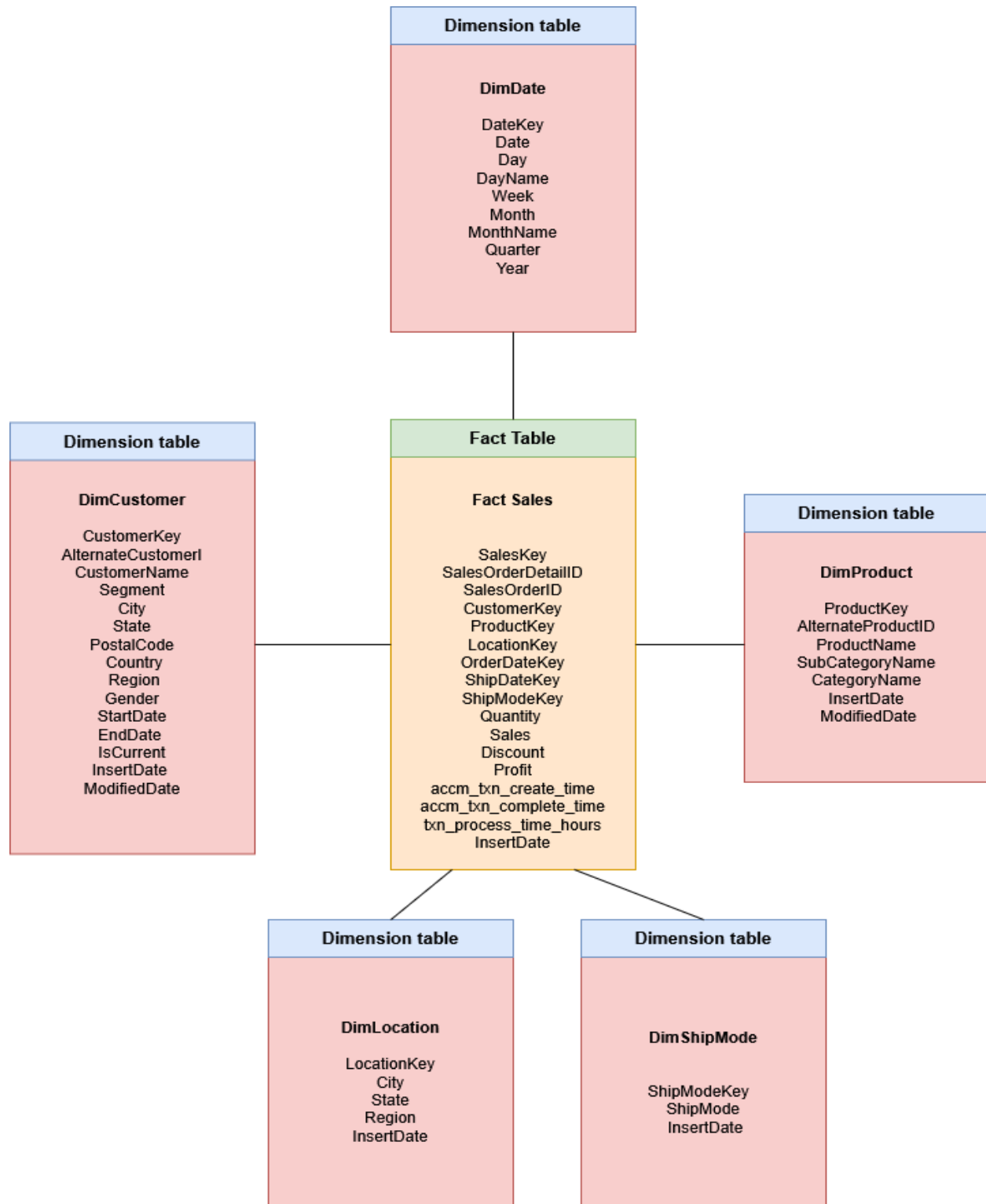


Figure : Star Schema Dimensional Model for Superstore_DW

4.3 Design Assumptions

- A star schema was chosen for its simplicity and superior query performance in analytical and reporting scenarios compared to a snowflake schema.
- The grain of FactSales is one row per individual order line item (RowID level from the OrderDetails source table).
- DimCustomer implements SCD Type 2 with StartDate, EndDate, and IsCurrent columns to preserve historical changes in the customer Segment field, as customers can move between Consumer, Corporate, and Home Office segments.
- DimDate was initially pre-populated with all dates from 1 January 2014 to 31 December 2017. It was subsequently extended to 31 December 2018 (1,826 total rows) to accommodate order and ship date records in the source data that extended beyond 2017, ensuring no rows were lost during the date lookup transformations in the ETL process.
- DimLocation, DimProduct, and DimShipMode are SCD Type 1 (overwrite) dimensions as changes to geography, product catalogue, or ship mode descriptions do not require historical tracking.
- All fact-to-dimension joins use surrogate keys generated during the ETL process, not natural keys from the source systems.
- AlternateCustomerID, AlternateProductID are stored in dimension tables for lineage tracing back to source systems.
- The three accumulating fact columns (accm_txn_create_time, accm_txn_complete_time, txn_process_time_hours) are added to FactSales as required by Task 6.

4.4 — SQL Implementation

The following SQL scripts were executed in SQL Server Management Studio (SSMS) to implement the Superstore_DW database and all dimension and fact tables.

4.4.1 — Create Superstore_DW Database

```
CREATE DATABASE Superstore_DW;  
GO  
USE Superstore_DW;  
GO
```

4.4.2 — Create DimDate

```
CREATE TABLE DimDate (  
    DateKey INT NOT NULL PRIMARY KEY,  
    Date DATE NOT NULL,  
    Day INT,  
    DayName VARCHAR(15),  
    Week INT,  
    Month INT,  
    MonthName VARCHAR(15),  
    Quarter INT,  
    Year INT  
);
```


4.4.3 — Create DimLocation

```
CREATE TABLE DimLocation (
    LocationKey INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    Region NVARCHAR(20) NOT NULL DEFAULT 'Unknown',
    State NVARCHAR(50) NOT NULL DEFAULT 'Unknown',
    City NVARCHAR(50) NOT NULL DEFAULT 'Unknown',
    InsertDate DATETIME DEFAULT GETDATE()
);
```

4.4.4 — Create DimShipMode

```
CREATE TABLE DimShipMode (
    ShipModeKey INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    ShipMode NVARCHAR(20) NOT NULL DEFAULT 'Unknown',
    InsertDate DATETIME DEFAULT GETDATE()
);
```

4.4.5 — Create DimProduct

```
CREATE TABLE DimProduct (
    ProductKey INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    AlternateProductID NVARCHAR(20) NOT NULL,
    ProductName NVARCHAR(255) NOT NULL,
    SubCategoryName NVARCHAR(50) NOT NULL DEFAULT 'Unknown',
    CategoryName NVARCHAR(50) NOT NULL DEFAULT 'Unknown',
    InsertDate DATETIME DEFAULT GETDATE(),
    ModifiedDate DATETIME DEFAULT GETDATE()
);
CREATE NONCLUSTERED INDEX IX_DimProduct_AlterID
ON DimProduct (AlternateProductID);
```

4.4.6 — Create DimCustomer (SCD Type 2)

```
CREATE TABLE DimCustomer (
    CustomerKey INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    AlternateCustomerID NVARCHAR(10) NOT NULL,
    CustomerName NVARCHAR(100) NOT NULL DEFAULT 'Unknown',
    Segment NVARCHAR(20) NOT NULL DEFAULT 'Unknown',
    City NVARCHAR(50),
    State NVARCHAR(50),
    PostalCode NVARCHAR(10),
    Country NVARCHAR(50),
    Region NVARCHAR(20),
    StartDate DATE NOT NULL DEFAULT CAST(GETDATE() AS DATE),
    EndDate DATE NULL,
    IsCurrent BIT NOT NULL DEFAULT 1,
    InsertDate DATETIME DEFAULT GETDATE(),
    ModifiedDate DATETIME DEFAULT GETDATE()
);
CREATE NONCLUSTERED INDEX IX_DimCustomer_Active
ON DimCustomer (AlternateCustomerID, IsCurrent);
```

4.4.7 — Create FactSales

```
CREATE TABLE FactSales (
    SalesKey INT NOT NULL IDENTITY(1,1) PRIMARY KEY,
    SalesOrderDetailID INT NOT NULL,
    SalesOrderID NVARCHAR(20),
    CustomerKey INT NOT NULL,
    ProductKey INT NOT NULL,
    LocationKey INT NOT NULL,
    OrderDateKey INT NOT NULL,
    ShipDateKey INT NOT NULL,
    ShipModeKey INT NOT NULL,
    Quantity INT NOT NULL DEFAULT 0,
    Sales DECIMAL(12,4) NOT NULL DEFAULT 0,
    Discount DECIMAL(5,4) NOT NULL DEFAULT 0,
    Profit DECIMAL(12,4) NOT NULL DEFAULT 0,
    accm_txn_create_time DATETIME NOT NULL DEFAULT GETDATE(),
    accm_txn_complete_time DATETIME NULL,
    txn_process_time_hours DECIMAL(10,2) NULL,
    InsertDate DATETIME DEFAULT GETDATE(),

    CONSTRAINT FK_Fact_Customer FOREIGN KEY (CustomerKey)
        REFERENCES DimCustomer(CustomerKey),
    CONSTRAINT FK_Fact_Product FOREIGN KEY (ProductKey)
        REFERENCES DimProduct(ProductKey),
    CONSTRAINT FK_Fact_Location FOREIGN KEY (LocationKey)
        REFERENCES DimLocation(LocationKey),
    CONSTRAINT FK_Fact_OrdDate FOREIGN KEY (OrderDateKey)
        REFERENCES DimDate(DateKey),
    CONSTRAINT FK_Fact_ShipDate FOREIGN KEY (ShipDateKey)
        REFERENCES DimDate(DateKey),
    CONSTRAINT FK_Fact_ShipMode FOREIGN KEY (ShipModeKey)
        REFERENCES DimShipMode(ShipModeKey)
);
CREATE NONCLUSTERED INDEX IX_FactSales_DetailID
ON FactSales (SalesOrderDetailID);
```

4.4.8 — Populate DimDate (2014–2018)

```
USE Superstore_DW;
DECLARE @d DATE = '2014-01-01';
DECLARE @e DATE = '2018-12-31';
WHILE @d <= @e
BEGIN
    INSERT INTO DimDate VALUES (
        CONVERT(INT, FORMAT(@d, 'yyyyMMdd')),
        @d,
        DAY(@d),
        DATENAME(WEEKDAY, @d),
        DATEPART(WEEK, @d),
        MONTH(@d),
        DATENAME(MONTH, @d),
        DATEPART(QUARTER, @d),
        YEAR(@d)
    );
    SET @d = DATEADD(DAY, 1, @d);
END
```

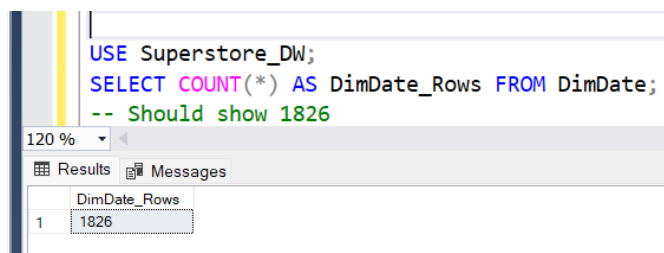


Figure : DimDate populated with 1,826 rows covering 2014–2018

4.4.9 — Stored Procedures for Dimension Loading

UpdateDimLocation

```
CREATE PROCEDURE dbo.UpdateDimLocation
    @Region NVARCHAR(20),
    @State  NVARCHAR(50),
    @City   NVARCHAR(50)
AS BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM DimLocation
        WHERE Region=@Region AND State=@State AND City=@City)
    INSERT INTO DimLocation (Region, State, City)
    VALUES (@Region, @State, @City);
END;
```

UpdateDimShipMode

```
CREATE PROCEDURE dbo.UpdateDimShipMode
    @ShipMode NVARCHAR(20)
AS BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM DimShipMode
        WHERE ShipMode = @ShipMode)
    INSERT INTO DimShipMode (ShipMode)
    VALUES (@ShipMode);
END;
```

UpdateDimProduct

```
CREATE PROCEDURE dbo.UpdateDimProduct
    @ProductID NVARCHAR(20),
    @ProductName NVARCHAR(255),
    @SubCategory NVARCHAR(50),
    @Category NVARCHAR(50)
AS BEGIN
    IF NOT EXISTS (
        SELECT 1 FROM DimProduct
        WHERE AlternateProductID = @ProductID)
    INSERT INTO DimProduct
        (AlternateProductID, ProductName,
        SubCategoryName, CategoryName,
        InsertDate, ModifiedDate)
    VALUES
        (@ProductID, @ProductName,
        @SubCategory, @Category,
        GETDATE(), GETDATE());
    ELSE
        UPDATE DimProduct
        SET ProductName = @ProductName,
            SubCategoryName = @SubCategory,
            CategoryName = @Category,
            ModifiedDate = GETDATE()
        WHERE AlternateProductID = @ProductID;
END;
```

4.4.10 — Verification Queries

After creating all tables and running all SSIS packages, the following queries were used to verify the implementation:

```
USE Superstore_DW;

SELECT 'DimDate' AS Table_Name, COUNT(*) AS Row_Count FROM DimDate
UNION ALL SELECT 'DimLocation', COUNT(*) FROM DimLocation
UNION ALL SELECT 'DimShipMode', COUNT(*) FROM DimShipMode
UNION ALL SELECT 'DimProduct', COUNT(*) FROM DimProduct
UNION ALL SELECT 'DimCustomer', COUNT(*) FROM DimCustomer
UNION ALL SELECT 'FactSales', COUNT(*) FROM FactSales;
```

109 %

Results Messages

	Table_Name	Row_Count
1	DimDate	1826
2	DimLocation	290
3	DimShipMode	4
4	DimProduct	1862
5	DimCustomer	793
6	FactSales	9994

```
USE Superstore_DW

SELECT
    SUM(CASE WHEN CustomerKey IS NULL THEN 1 ELSE 0 END) AS Null_CustomerKey,
    SUM(CASE WHEN ProductKey IS NULL THEN 1 ELSE 0 END) AS Null_ProductKey,
    SUM(CASE WHEN LocationKey IS NULL THEN 1 ELSE 0 END) AS Null_LocationKey,
    SUM(CASE WHEN OrderDateKey IS NULL THEN 1 ELSE 0 END) AS Null_OrderDateKey,
    SUM(CASE WHEN ShipDateKey IS NULL THEN 1 ELSE 0 END) AS Null_ShipDateKey,
    SUM(CASE WHEN ShipModeKey IS NULL THEN 1 ELSE 0 END) AS Null_ShipModeKey
FROM FactSales;
-- All should show 0
```

120 %

Results Messages

	Null_CustomerKey	Null_ProductKey	Null_LocationKey	Null_OrderDateKey	Null_ShipDateKey	Null_ShipModeKey
1	0	0	0	0	0	0

Figure: Verification query showing all DW table row counts and zero NULL surrogate keys in FactSales

Step 5: ETL Development

To populate the data warehouse with clean and integrated data, three ETL packages were developed **using SQL Server Integration Services (SSIS)**. The ETL process is divided into three main stages:

1. **Data Extraction to Staging** — extracting raw data from all three source types into staging tables.
2. **Transformation and Loading to the Data Warehouse** — transforming and loading clean data into dimensions and the fact table in the correct dependency order.
3. **Updating the Accumulating Fact Table** — updating the FactSales table with completion timestamps and derived process time fields.

Each package demonstrates key SSIS tasks such as connection setup, control and data flow design, transformations, and load operations from multiple data sources.

5.1 SSIS Packages Overview

Package Name	Purpose
Superstore_Load_Staging.dtsx	Extracts raw data from all three source types (SQL Server, Excel – Customers, Excel – Products) and loads into staging tables in Superstore_Staging database.
Superstore_Load_DW.dtsx	Transforms and loads clean data from staging into all dimension and fact tables in the correct dependency order using stored procedures, SCD, and Lookup transformations.
Superstore_UpdateFact.dtsx	Updates the accumulating fact table FactSales with completion timestamps (accm_txn_complete_time) and derives txn_process_time_hours.

Table — SSIS package overview

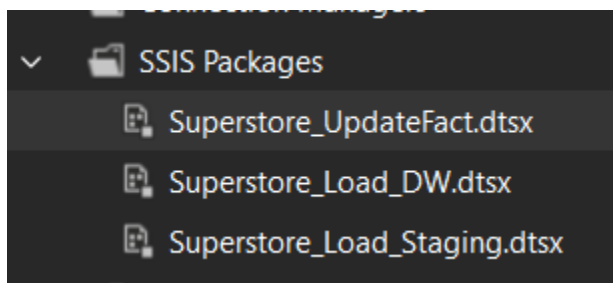


Figure : SSIS packages in Solution Explorer

5.2 Connection Managers

All ETL packages utilize multiple connection types to handle data from both databases and text files. The following connections were configured:

SQL Server – OLE DB Connections:

- Superstore_SourceDB — source OLTP database (Orders and OrderDetails tables)
- Superstore_Staging — intermediate staging database
- Superstore_DW — the data warehouse target database

Excel File Connections:

- Customers_Source.xlsx — Excel connection using the Microsoft Excel 2016 provider
- Products_Source.xlsx — Excel connection for product catalogue and hierarchy (two sheets)

These connection managers enable seamless integration between heterogeneous data sources.

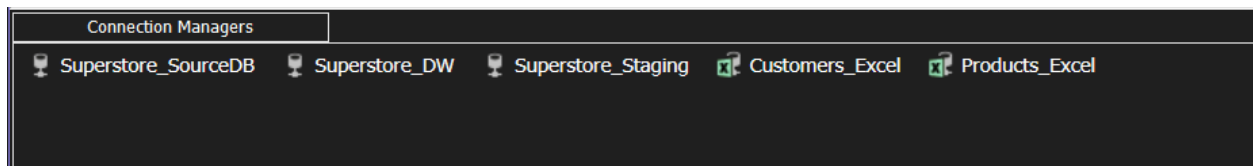


Figure : Connection Managers for all source types (3 OLE DB + 2 Excel).

5.3 Package 1: Superstore_Load_Staging.dtsx

This package extracts raw data from all three source systems and loads it into staging tables. Before data extraction begins, an Execute SQL Task named 'Truncate All Staging Tables' runs at the start of the Control Flow to truncate all five staging tables. This prevents duplicate data if the package is re-run. The control flow executes the following tasks sequentially:

#	Data Flow Task	Source	Staging Table
1	Extract Orders to Staging	Superstore_SourceDB — Orders table	StgOrders
2	Extract Order Details to Staging	Superstore_SourceDB — OrderDetails table	StgOrderDetails
3	Extract Customers to Staging	Customers_Source.xlsx (Excel)	StgCustomers
4	Extract Product Hierarchy to Staging	Products_Source.xlsx — Sheet: ProductHierarchy	StgProductHierarchy
5	Extract Products to Staging	Products_Source.xlsx — Sheet: Products	StgProducts

Table — Staging data flow tasks and their source-to-staging mapping

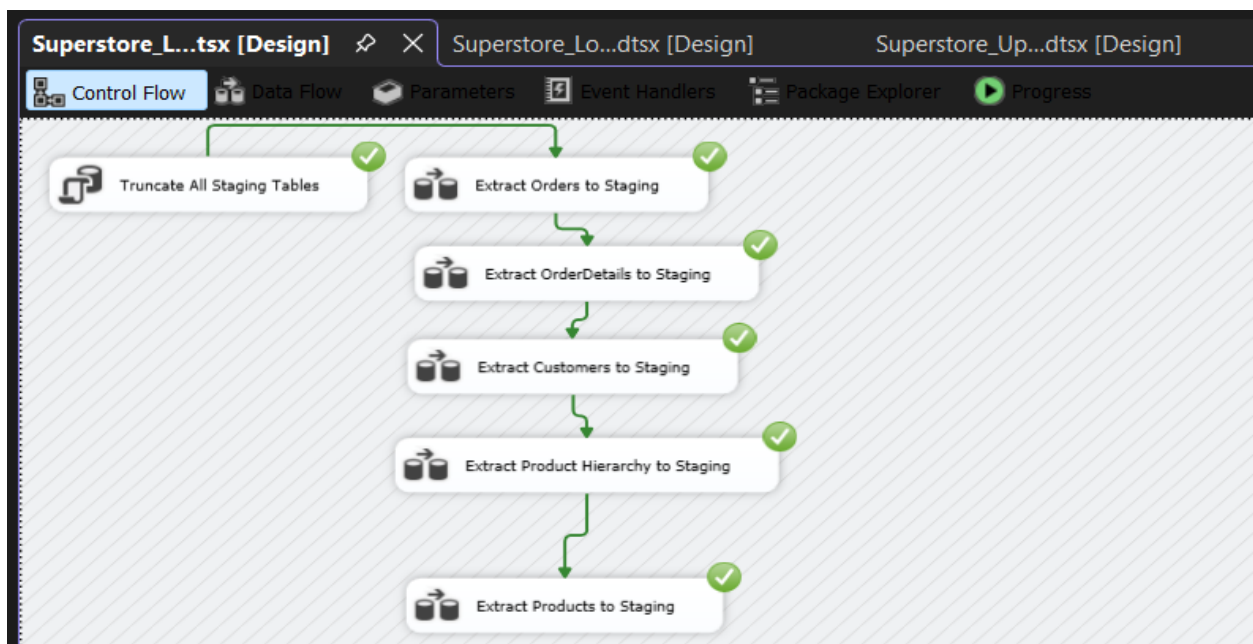


Figure —Control Flow in Superstore_Load_Staging.dtsx showing all five sequential staging tasks(Package 1)

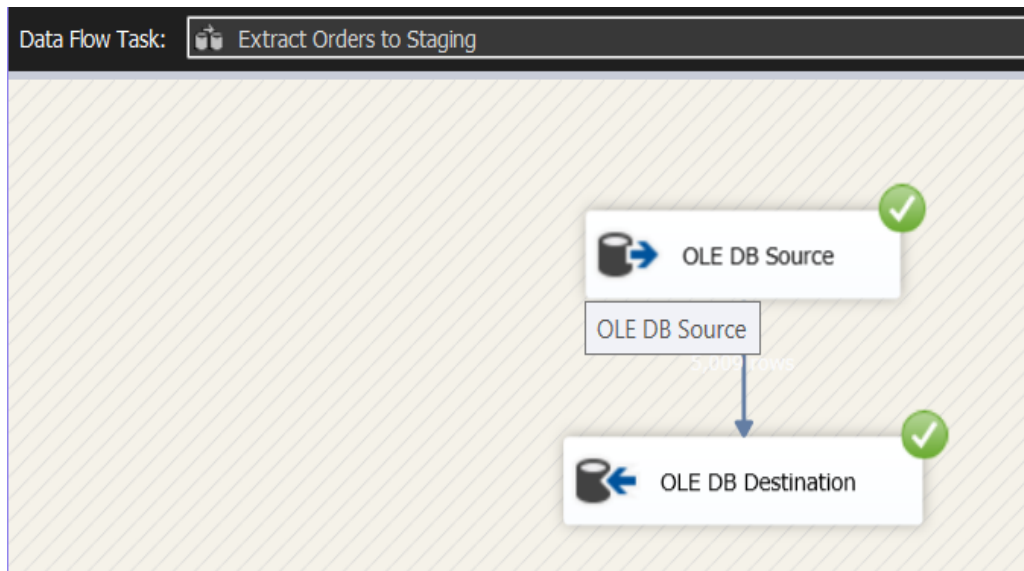


Figure : Orders extraction — 5,009 rows loaded to StgOrders

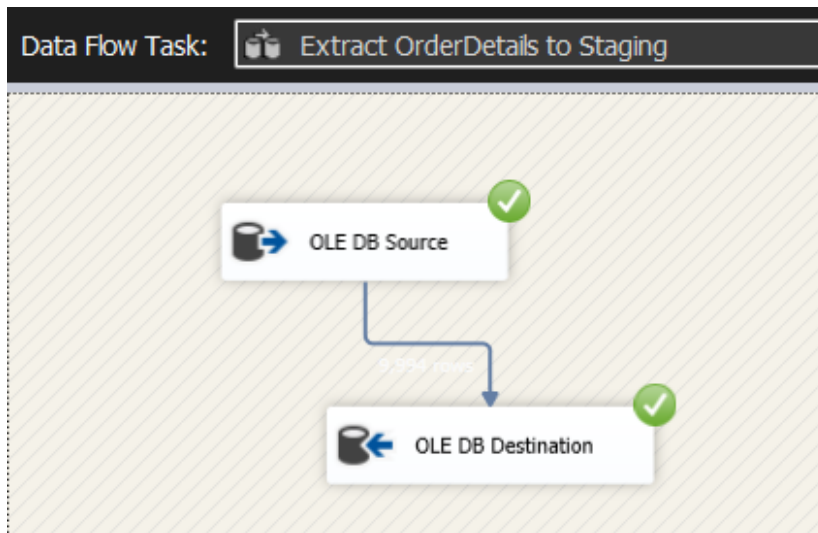


Figure : OrderDetails extraction — 9,994 rows loaded

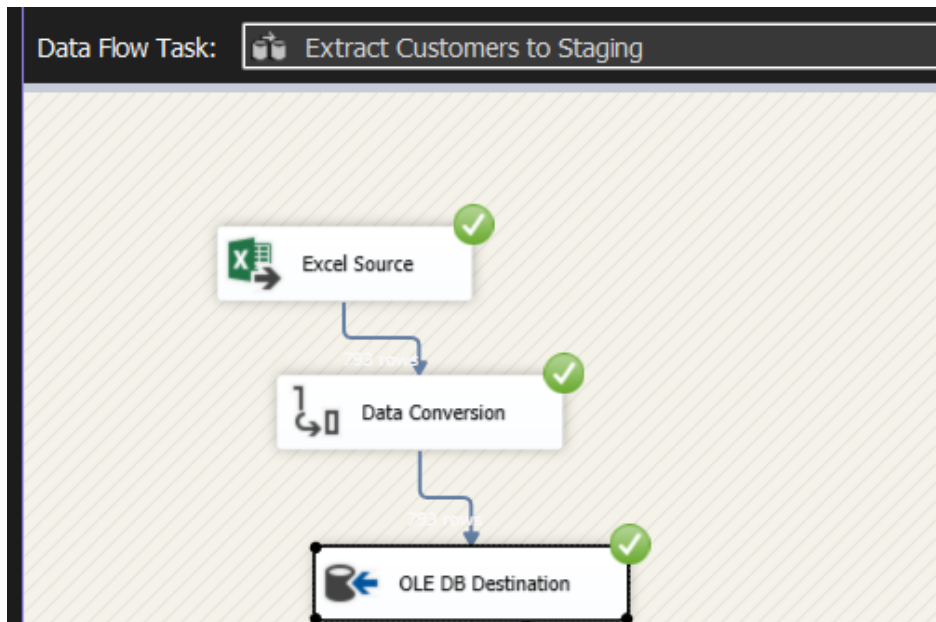


Figure : Customer extraction from Customers_Source.xlsx — 793 rows

A Data Conversion transformation was added between the Excel Source and OLE DB Destination to convert all columns from Unicode string (DT_WSTR length 255) returned by the Excel driver to the correct DW data types (DT_WSTR with appropriate lengths matching the staging table column definitions). This resolved truncation warnings caused by the Excel OLE DB provider returning all string columns at maximum length 255.

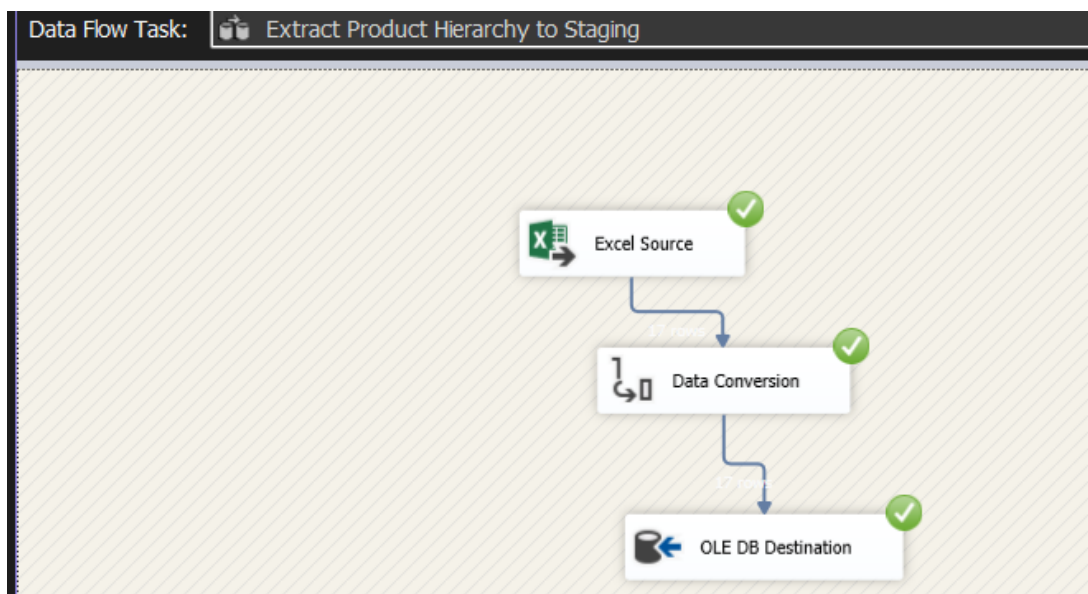


Figure : Product Hierarchy extraction — 17 rows loaded

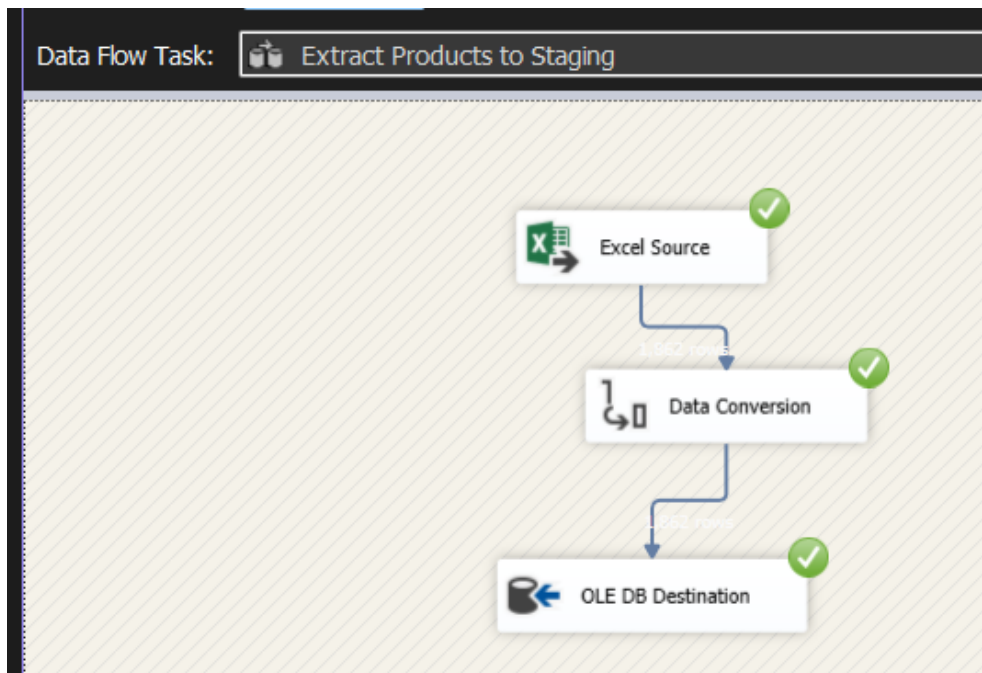


Figure : Products extraction — 1,862 rows loaded

```

SQLQuery2.sql - L...6EAMPD\USER (62))*) SQLQuery1.sql - L...6EAMPD\USER (67))*) Superstore_Source...6EAMPD\USER (64))
USE Superstore_Staging;
SELECT 'StgOrders' AS Table_Name, COUNT(*) AS Row_Count FROM StgOrders
UNION ALL
SELECT 'StgOrderDetails', COUNT(*) FROM StgOrderDetails
UNION ALL
SELECT 'StgCustomers', COUNT(*) FROM StgCustomers
UNION ALL
SELECT 'StgProductHierarchy', COUNT(*) FROM StgProductHierarchy
UNION ALL
SELECT 'StgProducts', COUNT(*) FROM StgProducts;
-- Expected: 5009 / 9994 / 793 / 17 / 1862
  
```

120 %

Results Messages

	Table_Name	Row_Count
1	StgOrders	5009
2	StgOrderDetails	9994
3	StgCustomers	793
4	StgProductHierarchy	17
5	StgProducts	1862

Figure: SSMS validation query showing all 5 staging tables with correct row counts

5.4 Package 2: Superstore_Load_DW.dtsx

This package handles the transformation and loading of clean data from staging tables into the dimensional and fact tables of the Superstore_DW. Tasks are executed in a strict dependency order to maintain referential integrity, dimensions must be fully loaded before the fact table is populated

#	Data Flow Task	Purpose
1	Transform and Load DimLocation	No FK dependencies. Loads unique City/State/Region combinations from StgCustomers.
2	Transform and Load DimShipMode	No FK dependencies. Loads unique ShipMode values from StgOrders.
3	Transform and Load DimProduct	Resolves SubCategoryID from StgProductHierarchy via Lookup; inserts/updates DimProduct using stored procedure UpdateDimProduct.
4	Transform and Load DimCustomer	SCD Type 2 — merges StgCustomers data; handles new and changed Segment records with StartDate/EndDate tracking.
5	Transform and Load FactSales	Performs Lookup transformations against DimDate, DimCustomer, DimProduct, DimLocation, and DimShipMode to resolve surrogate keys before loading FactSales.

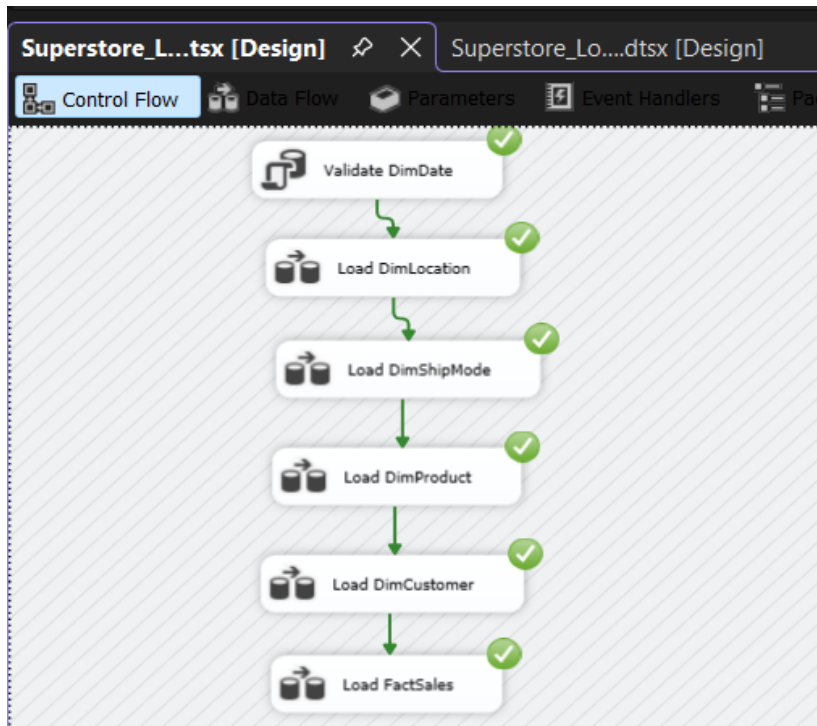


Figure — Control Flow in Superstore_Load_DW.dtsx showing the dimensional and fact loading sequence

5.4a Data Flow – Load DimLocation

This Data Flow reads all records from StgCustomers via an OLE DB Source. An OLE DB Source extracts distinct Region, State, and City combinations from StgCustomers using a SQL query with SELECT DISTINCT to eliminate duplicate location entries. An OLE DB Command executes the stored procedure UpdateDimLocation for each row, which performs an upsert, inserting the location if it does not already exist in DimLocation, or skipping if it does. An OLE DB Destination inserts records into DimLocation. Since location data does not require history tracking, a Type 1 (overwrite) approach via stored procedure UpdateDimLocation is used.

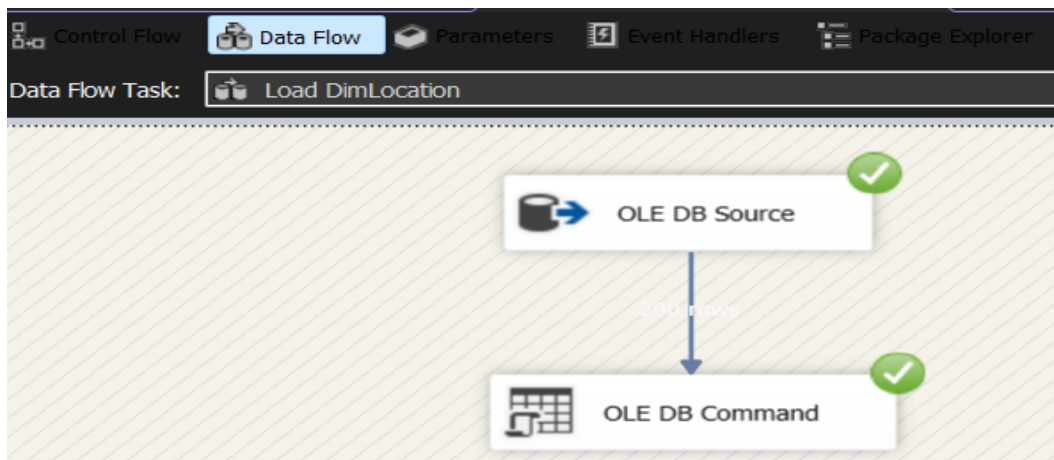


Figure —Data Flow for loading DimLocation from StgCustomers

5.4b Data Flow – Load DimShipMode

This Data Flow reads distinct ShipMode values from StgOrders via an OLE DB Source. The four valid values (Standard Class, Second Class, First Class, Same Day) are loaded directly into DimShipMode using a stored procedure UpdateDimShipMode which performs an upsert — inserting new ship modes or updating existing ones

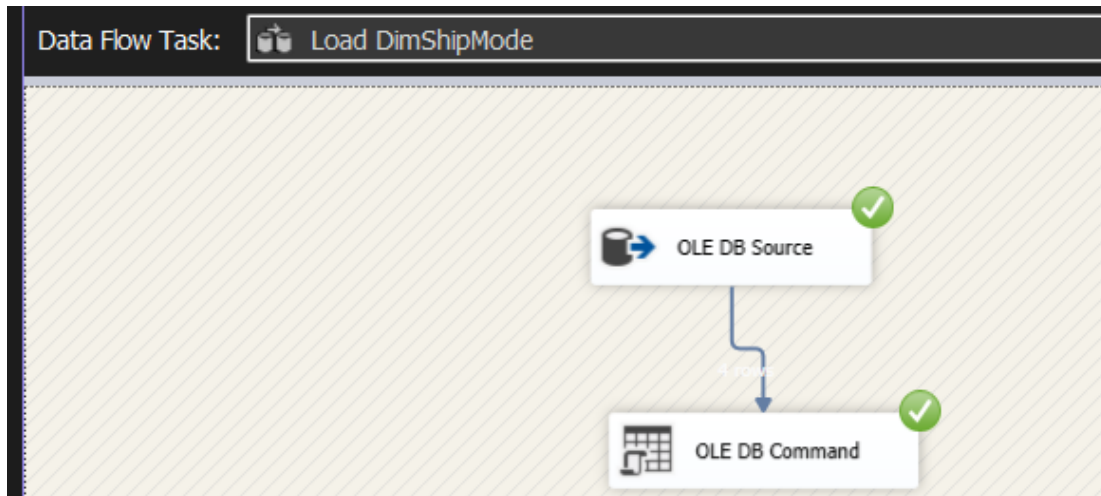


Figure — Data Flow for loading DimShipMode from StgOrders

5.4c Data Flow – Load DimProduct

This Data Flow extracts records from StgProducts using an OLE DB Source. A Lookup transformation reads SubCategory information from StgProductHierarchy using SubCategoryID to enrich product records with SubCategoryName before loading. An OLE DB Command then executes the stored procedure UpdateDimProduct for each row, performing an upsert based on AlternateProductID.

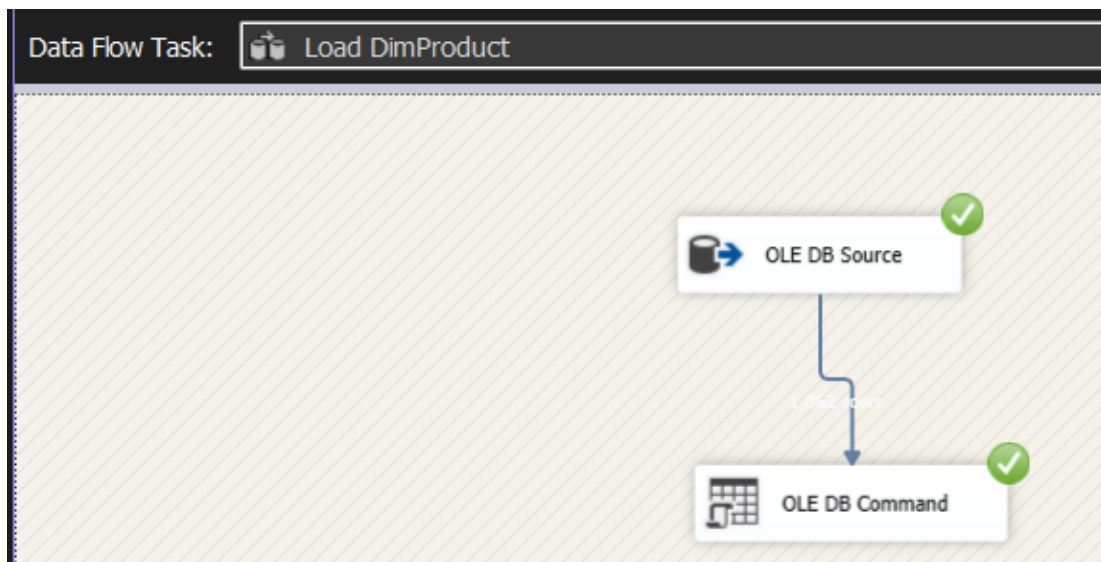


Figure — Data Flow for loading DimProduct with hierarchy lookup

5.4d Data Flow – Load DimCustomer (SCD Type 2)

This Data Flow implements Slowly Changing Dimension Type 2 logic for DimCustomer to track historical changes in the customer Segment field (Consumer, Corporate, Home Office). The process is as follows:

- An OLE DB Source reads all records from StgCustomers.
- A Derived Column transformation adds InsertDate and ModifiedDate using GETDATE().
- The Slowly Changing Dimension (SCD) component compares incoming records against existing DimCustomer records using AlternateCustomerID as the business key.
- New records (no match found) are routed to the Historical Attribute Inserts Output — a Derived Column sets StartDate = GETDATE(), EndDate = NULL, and IsCurrent = 1, before insertion via OLE DB Destination.
- Changed records (Segment has changed) are routed to the Historical Attribute Updates Output — an OLE DB Command closes the previous version by setting EndDate = GETDATE() and IsCurrent = 0, then the new version is inserted with a new StartDate.
- Unchanged records pass through without modification.

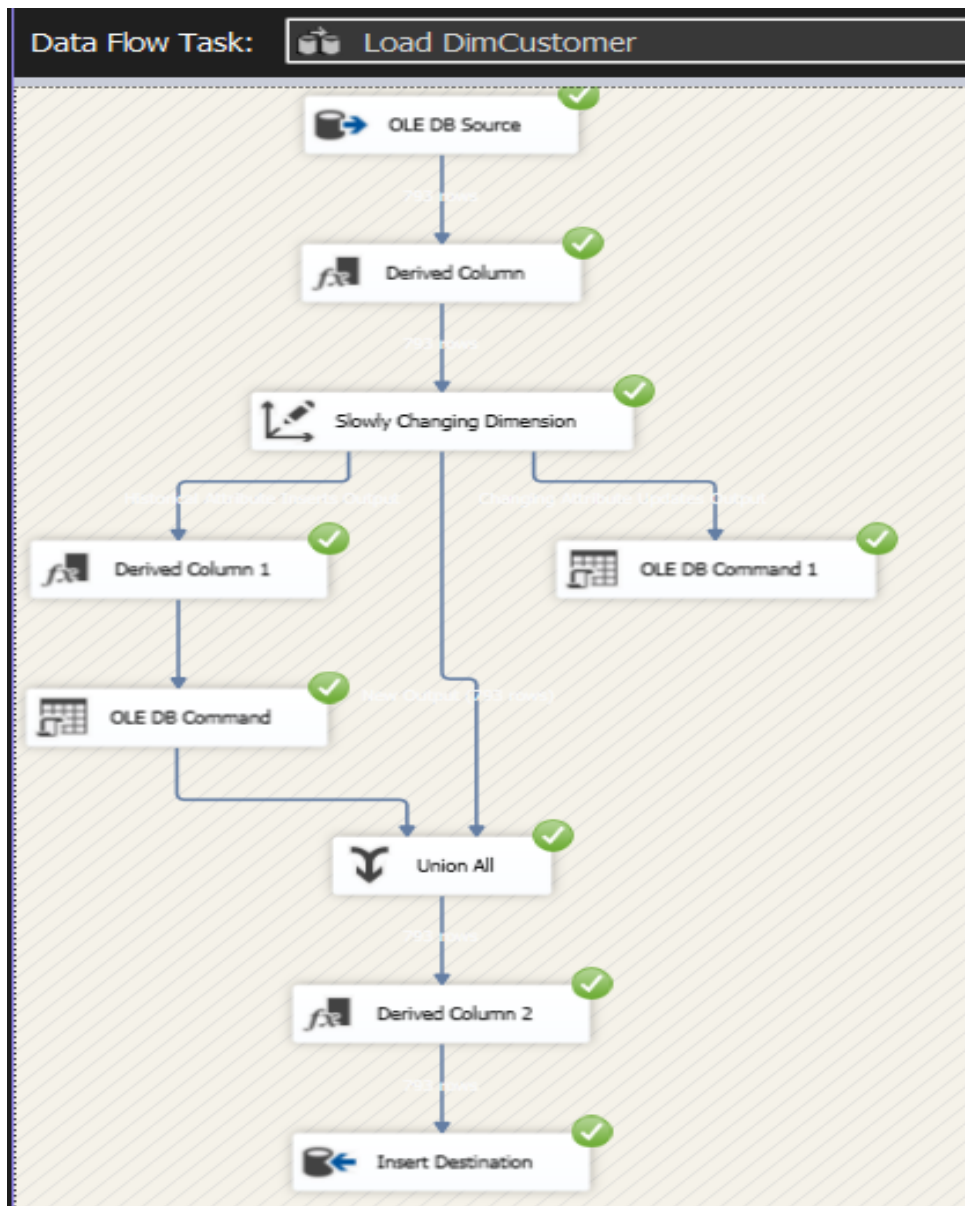


Figure — Data Flow for loading DimCustomer with SCD Type 2 logic

SQLQuery2.sql - L...6EAMPD\USER (62))* X SQLQuery1.sql - L...6E

```

USE Superstore_DW;
SELECT COUNT(*) FROM DimCustomer
  
```

120 %

Results Messages

	(No column name)
1	793

Figure: SSMS SELECT COUNT(*) FROM DimCustomer confirming all customer records loaded successfully

5.4e Data Flow – Load FactSales

Instead of using Sort and Merge Join components, a single OLE DB Source with a SQL JOIN query was used to combine StgOrderDetails, StgOrders, and StgCustomers. This approach is more efficient as the join is performed by the SQL Server query optimizer at the database level, reducing SSIS memory usage and component complexity. This is standard industry practice when source data resides in a relational staging database. It then performs a series of Lookup transformations to resolve surrogate keys from each dimension before inserting into FactSales:

Lookup Component	Input Column	DW Table	Output Alias
Order Date Lookup	OrderDate	DimDate	OrderDateKey
Ship Date Lookup	ShipDate	DimDate	ShipDateKey
Customer Lookup	CustomerID	DimCustomer	CustomerKey
Product Lookup	ProductID	DimProduct	ProductKey
Location Lookup	City + State	DimLocation	LocationKey
ShipMode Lookup	ShipMode	DimShipMode	ShipModeKey

After all surrogate keys are resolved, a Derived Column transformation adds `accm_txn_create_time = GETDATE()` for the accumulating fact. An OLE DB Destination then loads all resolved records into FactSales.



Figure — FactSales Data Flow showing all 6 Lookup components in sequence

<pre>USE Superstore_DW; SELECT COUNT(*) AS FactSales_Rows FROM FactSales;</pre>	
120 %	
Results	Messages
	FactSales_Rows
1	9994

Figure: SSMS SELECT COUNT(*) FROM FactSales showing 9994.

SQLQuery2.sql - L...6EAMPD\USER (62))* SQLQuery1.sql - L...6EAMPD\USER (67))* Superstore_Source...6EAMPD\USER (64))

-- Verify no NULL surrogate keys:
SELECT TOP 10
SalesKey, SalesOrderDetailID, CustomerKey, ProductKey,
LocationKey, OrderDateKey, ShipDateKey, ShipModeKey, Sales, Profit
FROM FactSales;
-- All FK columns should show integer values, not NULL

120 %

Results Messages

	SalesKey	SalesOrderDetailID	CustomerKey	ProductKey	LocationKey	OrderDateKey	ShipDateKey	ShipModeKey	Sales	Profit	
1	1	1	144	13	177	20161108	20161111	4	261.9600	41.9136	
2	2	2	144	56	177	20161108	20161111	4	731.9400	219.5820	
3	3	3	238	947	240	20160612	20160616	4	14.6200	6.8714	
4	4	4	706	320	158	20151011	20151018	3	957.5775	-383.0310	
5	5	5	706	1317	158	20151011	20151018	3	22.3680	2.5164	
6	6	6	89	186	240	20140609	20140614	3	48.8600	14.1694	
7	7	7	89	563	240	20140609	20140614	3	7.2800	1.9656	
8	8	8	89	1762	240	20140609	20140614	3	907.1520	90.7152	
9	9	9	89	795	240	20140609	20140614	3	18.5040	5.7825	
10	10	10	89	438	240	20140609	20140614	3	114.9000	34.4700	

Figure — SSMS SELECT TOP 10 * FROM FactSales confirming all surrogate keys are correctly populated (no NULLs)

Step 6 — ETL Development: Accumulating Fact Tables

6.1 Accumulating Fact Concept

An accumulating snapshot fact table tracks the lifecycle of a transaction across multiple stages. Unlike a transactional fact table where each row is inserted once and never updated, an accumulating snapshot fact table is updated as a transaction progresses through its stages. In this implementation, FactSales is extended with three additional columns to support this pattern:

Column	Description
accm_txn_create_time	Set to GETDATE() during initial FactSales load. Represents the system timestamp when the transaction record was first inserted into the data warehouse.
accm_txn_complete_time	Updated by the Superstore_UpdateFact.dtsx package when a matching RowID is found in the StgOrderCompletions staging table.
txn_process_time_hours	Derived field: DATEDIFF(HOUR, accm_txn_create_time, accm_txn_complete_time). Calculated by the UPDATE statement and stored persistently in the fact table.

Table — Accumulating fact columns added to FactSales for Task 6

6.2 OrderCompletions.csv

To simulate real-world transaction completion, a CSV file named OrderCompletions.csv was prepared containing 100 records. Each record contains the natural key of a FactSales row (RowID) and its corresponding completion timestamp. This file acts as the external data source for Package 3. The file was generated by running the following query in SSMS after Package 2 was loaded, ensuring the RowIDs exactly match SalesOrderDetailID values in FactSales:

```
SELECT TOP 100
    SalesOrderDetailID AS RowID,
    CONVERT(VARCHAR(19),
        DATEADD(HOUR, ABS(CHECKSUM(NEWID())) % 120 + 48,
            accm_txn_create_time), 120) AS accm_txn_complete_time
FROM FactSales
ORDER BY NEWID();
```

Figure: SSMS query used to generate OrderCompletions.csv — selecting 100 RowIDs with simulated completion timestamps.

```

RowID,accm_txn_complete_time
4227,2026-03-27 13:08:30
6819,2026-03-28 09:08:30
2292,2026-03-30 11:08:30
6257,2026-03-27 23:08:30
5354,2026-03-30 08:08:30
1481,2026-03-28 11:08:30
3470,2026-04-01 06:08:30
5591,2026-03-30 17:08:30
7943,2026-03-28 19:08:30
1366,2026-03-27 14:08:30
5863,2026-03-30 20:08:30
7736,2026-03-31 01:08:30
9603,2026-03-28 17:08:30
5416,2026-03-29 22:08:30
5506,2026-03-29 03:08:30
8665,2026-03-29 17:08:30
6225,2026-03-28 04:08:30
9984,2026-03-29 10:08:30
285,2026-03-28 22:08:30
8116,2026-03-29 16:08:30
1901,2026-03-31 14:08:30
7714,2026-03-29 17:08:30

```

Figure: OrderCompletions.csv open in Notepad showing RowID and accm_txn_complete_time columns

6.3 Package 3: Superstore_UpdateFact.dtsx

This package demonstrates the use of accumulating snapshot fact logic by updating the existing FactSales table with transaction completion data. The package control flow consists of the following sequential steps:

1. **Stage Completion Data** — A Data Flow Task reads OrderCompletions.csv and loads completion timestamps into the StgOrderCompletions staging table.
2. **Update FactSales** — An Execute SQL Task runs an UPDATE statement that populates accm_txn_complete_time and calculates txn_process_time_hours for matching rows.
3. **Truncate Staging Table** — An Execute SQL Task truncates StgOrderCompletions after the update to maintain a clean staging environment for future runs.

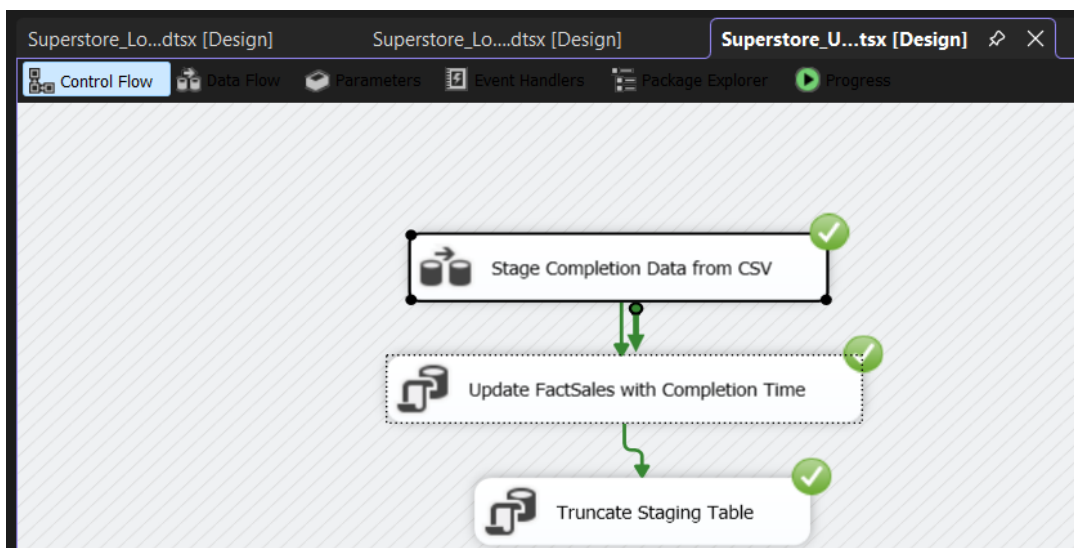


Figure —Control Flow for Superstore_UpdateFact.dtsx showing accumulating fact update logic(Package 3)

6.3a Data Flow — Stage Completion Data from CSV

This data flow task reads completed order records from the flat file source and stages them for the update operation. The purpose is to capture order completion timestamps which are later used to update the accumulating fact table.

- **Flat File Source:** Reads OrderCompletions.csv containing RowID and accm_txn_complete_time columns.
- **Derived Column Transformation:** Converts the accm_txn_complete_time field to the correct database timestamp format (DT_DBTIMESTAMP) to ensure compatibility with the FactSales column definition.
- **OLE DB Destination:** Loads the transformed records into StgOrderCompletions in Superstore_Staging, ready for the subsequent UPDATE operation.

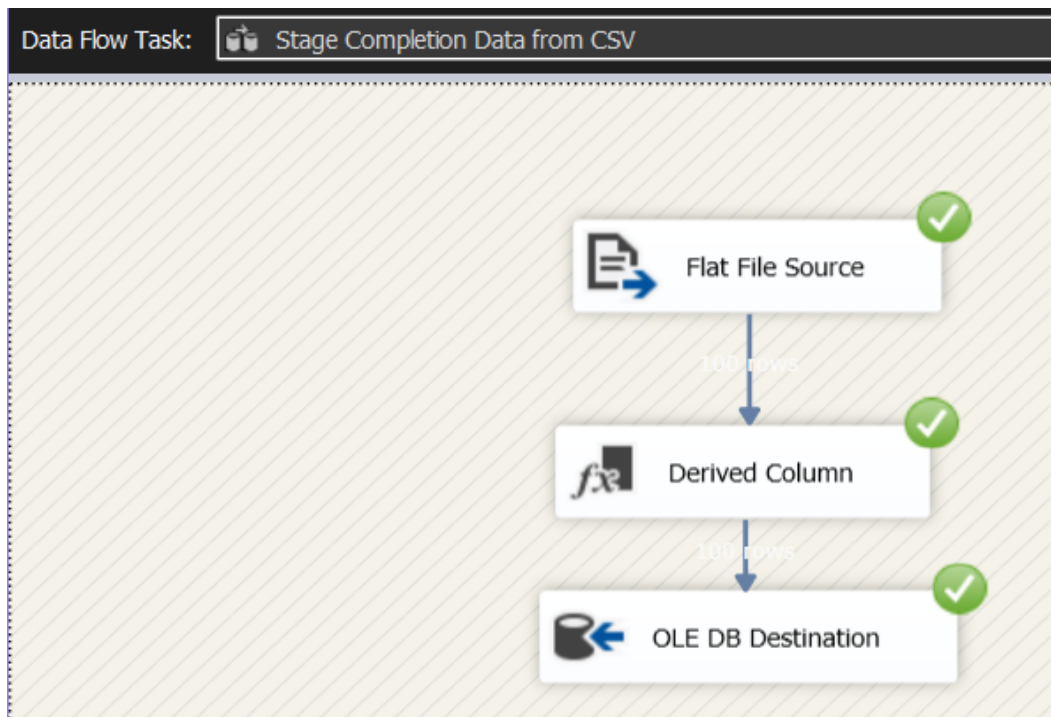


Figure: Package 3 Data Flow showing Flat File Source → Derived Column → OLE DB Destination

6.3b Execute SQL Task — Update FactSales with Completion Time

After staging the completion data, this Execute SQL Task runs the following UPDATE statement against Superstore_DW. It joins FactSales with StgOrderCompletions on SalesOrderDetailID = RowID and updates two fields:

- **accm_txn_complete_time:** The exact timestamp when the order was marked as completed, sourced from the staging table.
- **txn_process_time_hours:** A derived metric calculated as the total hours between accm_txn_create_time and accm_txn_complete_time using SQL Server's DATEDIFF function.

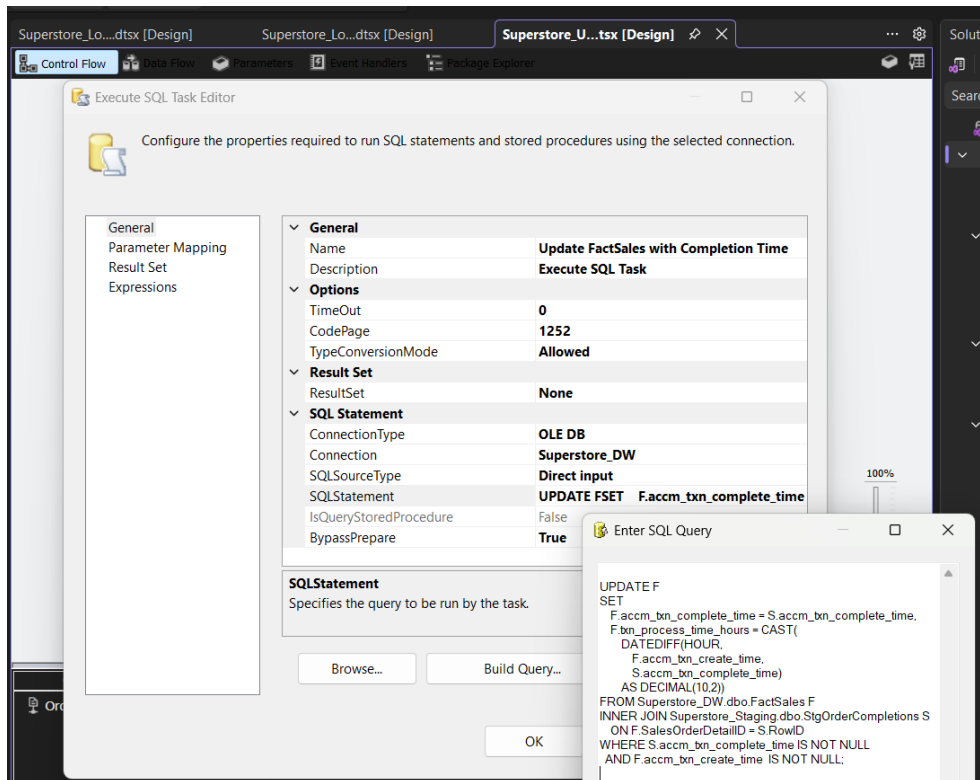


Figure : Execute SQL Task editor showing the UPDATE statement for accumulating fact update

The UPDATE statement includes two WHERE conditions to guard against invalid data: rows where accm_txn_complete_time is NULL in the staging table are skipped, and rows where accm_txn_create_time is NULL in FactSales are excluded. This ensures txn_process_time_hours is only calculated when both timestamps are valid, preventing incorrect derived values.

6.3c Execute SQL Task — Truncate Staging Table

After the UPDATE is completed successfully, a final Execute SQL Task truncates the StgOrderCompletions staging table. This ensures the staging environment is clean for subsequent runs of Package 3 and prevents duplicate updates on future executions.

```
TRUNCATE TABLE StgOrderCompletions;
```

6.4 Validation Results

After running Package 3, the following validation query was executed in SSMS to confirm that 100 rows were successfully updated with completion timestamps and that `txn_process_time_hours` was correctly calculated:

```
SQLQuery2.sql - L...6EAMPD\USER (62))*  SQLQuery1.sql - L...6EAMPD\USER (67))*  Superstore_Source...6EAMPD\USER (64))
USE Superstore_DW;
SELECT
    COUNT(*) AS Total_Rows,
    SUM(CASE WHEN accm_txn_complete_time IS NOT NULL THEN 1 ELSE 0 END) AS Completed,
    SUM(CASE WHEN accm_txn_complete_time IS NULL THEN 1 ELSE 0 END) AS Pending,
    ROUND(AVG(txn_process_time_hours),1) AS Avg_Process_Hours
FROM FactSales;
-- Expected: Completed=100, Pending=9894, Avg_Process_Hours~80
-- Show the actual updated rows:
SELECT TOP 10
    SalesOrderDetailID,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM FactSales
WHERE accm_txn_complete_time IS NOT NULL
ORDER BY SalesOrderDetailID;
```

Figure : Validation query

Results				
	Total_Rows	Completed	Pending	Avg_Process_Hours
1	9994	100	9894	99.500000

	SalesOrderDetailID	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	91	2026-03-25 09:08:30.350	2026-03-28 10:08:30.000	73.00
2	108	2026-03-25 09:08:30.350	2026-03-29 00:08:30.000	87.00
3	161	2026-03-25 09:08:30.350	2026-03-28 11:08:30.000	74.00
4	260	2026-03-25 09:08:30.350	2026-03-31 10:08:30.000	145.00
5	285	2026-03-25 09:08:30.350	2026-03-28 22:08:30.000	85.00
6	381	2026-03-25 09:08:30.350	2026-03-30 16:08:30.000	127.00
7	489	2026-03-25 09:08:30.350	2026-03-29 05:08:30.000	92.00
8	661	2026-03-25 09:08:30.350	2026-03-28 20:08:30.000	83.00
9	667	2026-03-25 09:08:30.350	2026-03-31 21:08:30.000	156.00
10	949	2026-03-25 09:08:30.350	2026-03-29 11:08:30.000	98.00

Query executed successfully.

Figure: SSMS query result showing `OrderID`, `accm_txn_create_time`, `accm_txn_complete_time`, and `txn_process_time_hours` — all 100 updated rows showing correct positive hour values confirming Package 3 executed successfully

7. Conclusion

This assignment successfully designed and implemented a complete Data Warehouse and Business Intelligence solution for the Superstore Retail Sales dataset, covering all six tasks specified in the IT3021 Assignment 1 brief.

The solution integrates data from three heterogeneous source types — SQL Server database, and two Microsoft Excel files — into a Star Schema dimensional model consisting of one fact table (FactSales) and five dimension tables across the Superstore_DW database. The ETL pipeline was built using three SSIS packages that enforce correct loading order, apply data type transformations, implement SCD Type 2 history tracking on DimCustomer, and resolve all surrogate key lookups with zero NULL violations across 9,994 fact rows.

The accumulating snapshot fact pattern was fully implemented through Package 3, which reads completion timestamps from an external CSV file and updates FactSales with accm_txn_complete_time and a derived txn_process_time_hours field, demonstrating real-world transaction lifecycle management within a data warehouse.

The Superstore_DW is designed and validated to serve as the foundation for Assignment 2, where it will be consumed by an SSAS multidimensional cube and Power BI reports for analytical reporting.